

Mobile Application Development

Unit No. 05

Android Application Deployment

What is Android Application Deployment :-

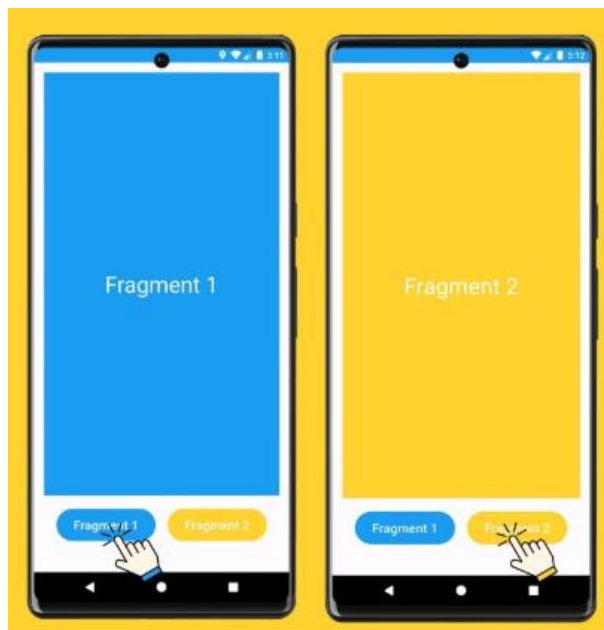
- Android Application Deployment is the final phase of development where a built, signed, and tested app (using APK or Android App Bundle format) is released, installed, and made available to users.
- Android Application Deployment is the process of making an app available to users.
- The primary methods are publishing to the Google Play Store for public distribution.

5.1– Advanced Concepts:-

- Advanced Concept in Android development creates a wide range of topics that move beyond the basics of creating simple applications.
- These concepts focus on building robust, performing, scalable, maintainable applications.

5.1.1 – Fragments:-

- A **Fragment** in Android is a **reusable part of a user interface (UI)** that lives inside an **Activity**.
- Fragment is a part of **Activity**. It is also known as sub-activity.
- It defines and manages its own layout, has its own lifecycle, and can handle its own input events, providing flexibility in designing UIs for different screen sizes and orientations.



5.1.1 - Fragment

Example:-

1 - fragment_my.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, This is a Fragment"
        android:textSize="20sp"/>
</LinearLayout>
```

2 - MyFragment.java

```
package com.example.fragmentexample;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.fragment.app.Fragment;

public class MyFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState) {

        // Connect fragment layout
        return inflater.inflate(R.layout.fragment_my, container, false);
    }
}
```

3 – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

4 - MainActivity.java

```
package com.example.fragmentexample;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentTransaction;

public class MainActivity extends AppCompatActivity {

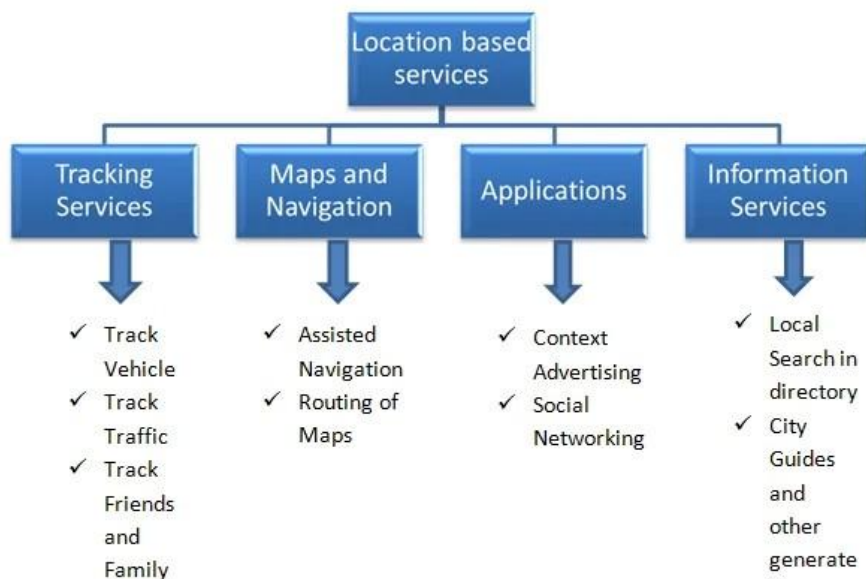
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Add fragment to activity
        MyFragment fragment = new MyFragment();

        FragmentTransaction transaction = getSupportFragmentManager().beginTransaction();
        transaction.add(R.id.fragment_container, fragment);
        transaction.commit();
    }
}
```

5.1.2 - Location based services:-

- **Location-Based Services (LBS)** are mobile app features that use a device's geographic position to provide services such as navigation, nearby places and tracking.
- LBS app follows the location and offer extra services such as locating facilities close at hand presenting suggestions for route and so on.
- In location based service application map is the key components, which present visual location of our location.



5.2 – Location Based Service

5.1.3 - SMS telephony :-

- Android device can send and receive message to or from any other phone that supports Short Message Service (SMS).
- It's the part of Android that lets your phone **send, receive, store, and manage SMS text messages** using your SIM card and mobile carrier.

➤ What SMS Telephony Does:-

1. Sending text messages.
2. Receiving text messages.
3. Communicating with your mobile carrier (Wireless Service Provider)
4. Managing SIM-based messaging services.
5. Storing SMS in the device database.
6. Triggering notifications when messages arrive.

• Simple Send SMS Program:-

1 – activity_main.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:orientation="vertical"
android:padding="20dp"
android:layout_width="match_parent"
android:layout_height="match_parent">
```

```
<EditText
android:id="@+id/phoneNumber"
android:hint="Enter phone number"
android:inputType="phone"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
```

```
<EditText
android:id="@+id/message"
android:hint="Enter message"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
```

```
<Button
android:id="@+id/sendBtn"
android:text="Send SMS"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
</LinearLayout>
```

2 – MainActivity.java

```
package com.example.sendsms;

import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.telephony.SmsManager;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

public class MainActivity extends AppCompatActivity {

    EditText phoneNumber, message;
    Button sendBtn;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        phoneNumber = findViewById(R.id.phoneNumber);
        message = findViewById(R.id.message);
        sendBtn = findViewById(R.id.sendBtn);

        sendBtn.setOnClickListener(v -> sendSMS());
    }
    private void sendSMS() {
        String number = phoneNumber.getText().toString();
        String msg = message.getText().toString();

        // Request permission if not granted
        if (ActivityCompat.checkSelfPermission(this,
            Manifest.permission.SEND_SMS) != PackageManager.PERMISSION_GRANTED) {

            ActivityCompat.requestPermissions(this,
                new String[]{Manifest.permission.SEND_SMS}, 1);
            return;
        }
        SmsManager smsManager = SmsManager.getDefault();
        smsManager.sendTextMessage(number, null, msg, null, null);

        Toast.makeText(this, "SMS Sent!", Toast.LENGTH_SHORT).show();
    }
}
```

3 – AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.example.sendsms">

<!-- Permission to send SMS -->
<uses-permission android:name="android.permission.SEND_SMS"/>

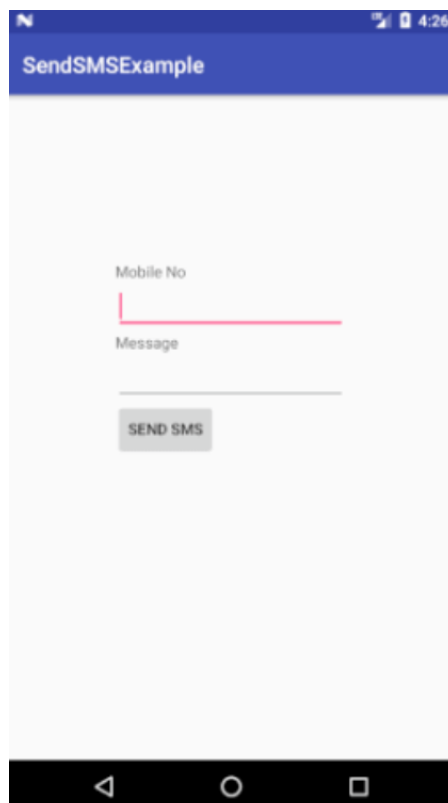
<application
android:allowBackup="true"
android:label="SendSMS"
android:icon="@mipmap/ic_launcher"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportRtl="true"
android:theme="@style/Theme.AppCompat.Light.DarkActionBar">

<activity android:name=".MainActivity">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>

</application>

</manifest>
```

4 – Output:-



5.1.4 – Audio Capture:-

- In android for recording audio or video, there is a built-in class called **MediaRecorder**.
- This class helps to easily record video and audio files.
- Android has a built-in microphone through which you can capture audio and store it, or play it in your phone.

Program for Audio Capture:-

1 - activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_gravity="center"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btnStart"
        android:text="Start Recording"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnStop"
        android:text="Stop Recording"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

2 - MainActivity.java

```
package com.example.audiorecorder;

import android.media.MediaRecorder;
import android.os.Bundle;
import android.os.Environment;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
import java.io.IOException;

public class MainActivity extends AppCompatActivity {
    private MediaRecorder recorder;
    private String fileName;
    @Override
```

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Button start = findViewById(R.id.btnStart);
    Button stop = findViewById(R.id.btnStop);
    fileName = Environment.getExternalStorageDirectory().getAbsolutePath() + "/recorded_audio.3gp";
    start.setOnClickListener(v -> startRecording());
    stop.setOnClickListener(v -> stopRecording());
}

private void startRecording() {
    recorder = new MediaRecorder();
    recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
    recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
    recorder.setOutputFile(fileName);
    recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
    try {
        recorder.prepare();
        recorder.start();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void stopRecording() {
    if (recorder != null) {
        recorder.stop();
        recorder.release();
        recorder = null;
    }
}
}

```

3 - AndroidManifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.audiorecorder">

    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

    <application
        android:allowBackup="true"
        android:label="AudioRecorder"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>

```


5.1.5 – Camera:-

- Android provides APIs to **access the device camera**, capture photos, record videos, and process frames.
- There are two ways to integrate the camera module.

Develop a Camera Application:-

1 – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/buttonTakePhoto"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Take Photo"/>

    <ImageView
        android:id="@+id/imageViewPhoto"
        android:layout_width="300dp"
        android:layout_height="300dp"
        android:layout_marginTop="20dp"
        android:scaleType="centerCrop"
        android:background="#ddd"/>
</LinearLayout>
```

2 – MainActivity.java

```
package com.example.simplecameraapp;

import android.app.Activity;
import android.content.Intent;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.provider.MediaStore;
import android.widget.Button;
import android.widget.ImageView;
import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
```

```

private static final int CAMERA_REQUEST_CODE = 100;
private ImageView imageView;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Button buttonTakePhoto = findViewById(R.id.buttonTakePhoto);
    imageView = findViewById(R.id.imageViewPhoto);
    buttonTakePhoto.setOnClickListener(v -> {
        Intent cameraIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
        startActivityForResult(cameraIntent, CAMERA_REQUEST_CODE);
    });
}
@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == CAMERA_REQUEST_CODE && resultCode == Activity.RESULT_OK) {
        Bitmap photo = (Bitmap) data.getExtras().get("data");
        imageView.setImageBitmap(photo);
    }
}
}

```

3 – AndroidManifest.xml

```

<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

```

5.1.6 - Bluetooth:-

- **Bluetooth** is a **wireless technology standard** for exchanging data over short distances (usually up to 10 meters for most devices) using **radio waves** in the **2.4 GHz ISM** (Industrial, Scientific, and Medical radio) **band**.
- It allows devices like **phones, laptops, headphones, speakers, smartwatches, and cars** to communicate **without wires**.

❖ Bluetooth Version:-

Version	Year	Key Features
1.0 / 1.0B	1999	Basic connectivity, slow (1 Mbps)
2.0 + EDR	2004	Faster (3 Mbps), lower power
3.0 + HS	2009	High-speed data transfer (up to 24 Mbps with Wi-Fi)
4.0	2010	Bluetooth Low Energy (BLE) for IoT, wearables
5.0	2016	Longer range (4x), faster speed, more broadcasting capacity
5.1 / 5.2	2019	Direction finding, improved audio, low latency

❖ Develop a Simple Bluetooth Device:-

1 – activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/b1"
        android:text="Bluetooth ON"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/b2"
        android:text="Bluetooth OFF"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

2 – MainActivity.java

```
package com.example.myapplication;

import android.Manifest;
import android.bluetooth.BluetoothAdapter;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

public class MainActivity extends AppCompatActivity {

    BluetoothAdapter bt;
    Button on, off;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```

super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

on = findViewById(R.id.b1);
off = findViewById(R.id.b2);

bt = BluetoothAdapter.getDefaultAdapter();

// Request permission for Android 12+
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.BLUETOOTH_CONNECT) !=
PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.BLUETOOTH_CONNECT},
1);
    }
}

// Bluetooth ON
on.setOnClickListener(v -> {
    if (bt == null) {
        Toast.makeText(this, "Bluetooth not supported",
Toast.LENGTH_SHORT).show();
    } else if (!bt.isEnabled()) {
        Intent intent = new
Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
        startActivity(intent);
    } else {
        Toast.makeText(this, "Already ON", Toast.LENGTH_SHORT).show();
    }
});

// Bluetooth OFF
off.setOnClickListener(v -> {
    if (bt != null && bt.isEnabled()) {
        bt.disable();
        Toast.makeText(this, "Bluetooth OFF",
Toast.LENGTH_SHORT).show();
    }
});
}
}

```

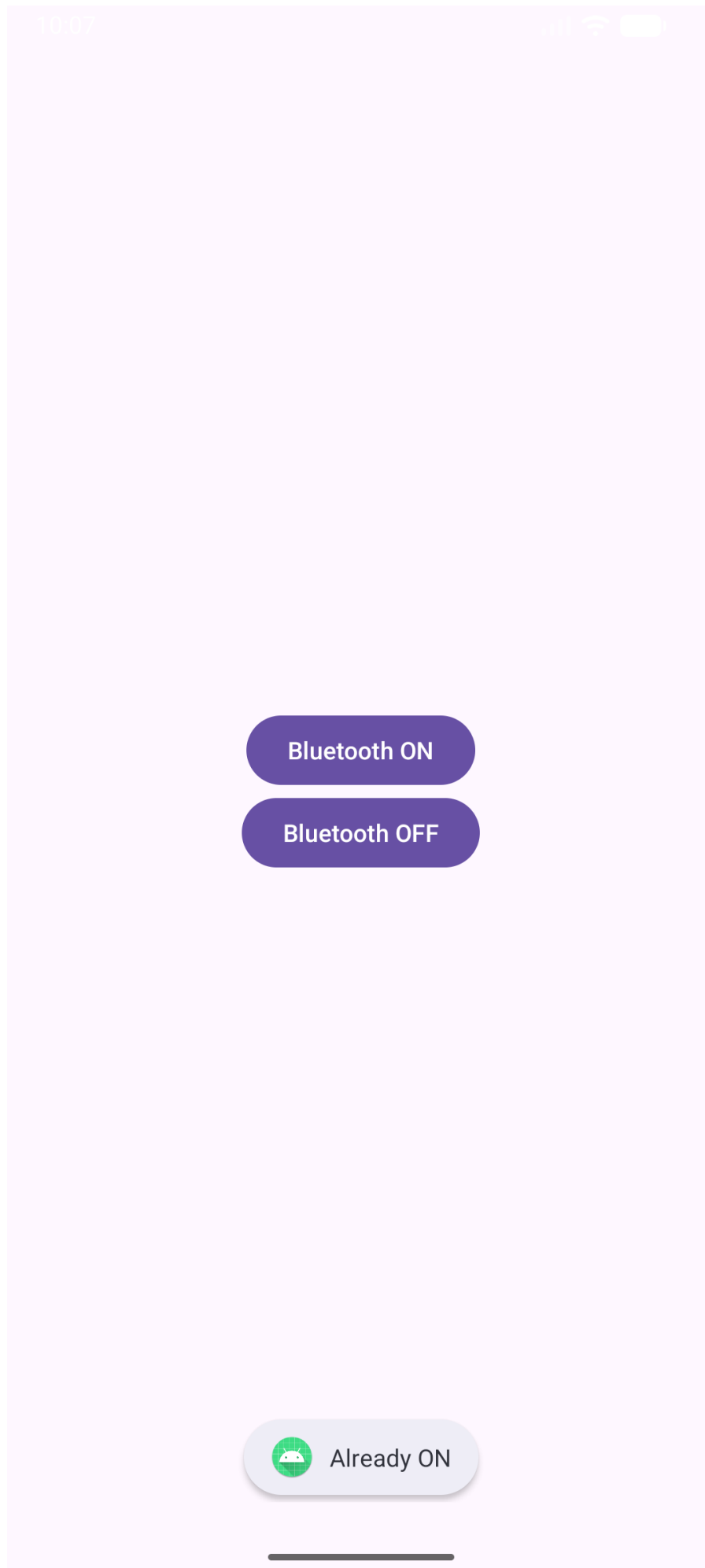
3 – AndroidManifest.xml

```

<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>

```

- **Output:-**



5.2 - Security Concepts

5.2.1 - Android security model:-

- Android is a multi-process system, in which each application (and parts of the system) runs in its own process.
- The Android Security Model is a layered system designed to protect user data, apps, and the device itself.
- It combines operating system features, app-level controls, and user permissions to keep everything secure.
- There are three protection levels that affect third-party apps, normal, signature permission and dangerous permission.

1 – Normal Permission:-

- These are low-risk permissions that do not affect user privacy or device operation.
- Ex:- Accessing the internet, Setting time zone, Vibrating the device, etc.

2 - Dangerous Permissions:-

- These involve sensitive user data or critical device features.
- Ex:- Camera, Location, Contacts, Microphone, Storage access, etc.

3 - Signature Permissions:-

- These are **highly secure permissions** used for communication between apps from the same developer.
- Ex:- System-level app interactions, Access between apps from the same company (e.g., Google apps), Banking Sector, etc.

5.2.2 - Declaring and using permissions:-

- **Declaring and using permissions** means telling the system what permissions your app needs, and actually requesting and using them safely in code.
- **Example:-**
<manifest ... >

```
<uses-permission android:name="android.permission.INTERNET"/>
```

```
<uses-permission android:name="android.permission.CAMERA"/>
```

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

```
</manifest>
```

5.2.3 - Using custom permission:-

- A custom permission is a permission you create in your app to protect your app's data/components and Control which apps can interact with your app.

- Example:-

```
<manifest ... >
```

```
<!-- Define custom permission -->
```

```
<permission
```

```
  android:name="com.example.myapp.MY_PERMISSION"
```

```
  android:label="My Custom Permission"
```

```
  android:description="Allows access to my app features"
```

```
  android:protectionLevel="dangerous"/>
```

```
</manifest>
```

- **Different types of Custom Permission:-**

1 - Protect an Activity:-

```
<activity
```

```
  android:name=".MyActivity"
```

```
  android:permission="com.example.myapp.MY_PERMISSION">
```

```
</activity>
```

2 - Protect a Service:-

```
<service
```

```
  android:name=".MyService"
```

```
  android:permission="com.example.myapp.MY_PERMISSION">
```

```
</service>
```

3 - Protect a Broadcast Receiver:-

```
<receiver
```

```
  android:name=".MyReceiver"
```

```
  android:permission="com.example.myapp.MY_PERMISSION">
```

```
</receiver>
```

5.3 - Application Deployment :-

5.3.1 - Process for creating and deploying Android applications on Google Play store, become a Publisher :-

Step 1 - Install Android Studio :-

This software provides all tools needed to develop, run, and test Android apps.

Step 2 - Install SDK & Emulator :-

SDK gives required libraries, emulator acts like a virtual Android phone.

Step 3 - Create New Project (Java) :-

Start a new app project and select Java as the programming language.

Step 4 - Understand Project Structure :-

MainActivity.java handles logic and activity_main.xml handles design.

Step 5 - Design UI (XML) :-

Add buttons, text, images to define how the app looks.

Step 6 - Write Java Code :-

Add functionality like button click actions using Java.

Step 7 - Connect UI with Code :-

Use findViewById() to link XML elements with Java code.

Step 8 - Run the App :-

Execute the app on emulator or real device.

Step 9 - Test the App :-

Check all features and fix bugs or errors.

Step 10 - Debug & Optimize :-

Improve performance and remove crashes using Logcat.

Step 11 - Generate Signed AAB :-

Create a release version of the app for publishing.

Step 12 - Create Keystore :-

Generate a secure key used to sign your app (needed for updates).

Step 13 - Register on Google Play Console :-

Create a developer account by paying a one-time fee.

Step 14 - Create App Listing :-

Enter app name, category, and basic information.

Step 15 - Add Store Details :-

Upload app icon, screenshots, and write description.

Step 16 - Fill App Content Section :-

Provide privacy policy, permissions, and content rating.

Step 17 - Follow Google Play Developer Policies :-

Ensure your app meets all rules to avoid rejection.

Step 18 - Upload AAB File :-

Submit your app file to Play Console.

Step 19 - Submit for Review :-

Google checks your app for quality and security.

Step 20 - Publish App :-

After approval, your app goes live on the Play Store.

Step 21 - Monitor Performance :-

Track downloads, ratings, and crashes.

Step 22 - Update App Regularly :-

Fix bugs and add new features over time.